Virtual Machine Firewall Implementation

Understanding Protocols, Ports, Firewall Policies & Rule Implementation

Estimated Time: **7–14 hours**

1. Project Overview

In this hands-on lab, you will set up a Linux virtual machine from scratch, then configure a host-based firewall to control incoming and outgoing network traffic. The project is designed to give you practical experience with the concepts that underpin every enterprise network: protocols, ports, firewall policies, and rule-based packet filtering.

By the end of this lab you will have a working firewall that enforces a real-world security policy you designed yourself.

2. Learning Objectives

After completing this project, you will be able to:

- Explain the role of common network protocols (TCP, UDP, ICMP) and their associated port numbers.
- Differentiate between well-known ports (0–1023), registered ports (1024–49151), and dynamic ports (49152–65535).
- Describe what a firewall policy is and why a default-deny approach is preferred over default-allow.
- Use both UFW and iptables to create, modify, and verify firewall rules on a Linux system.
- Test firewall rules using standard network diagnostic tools (ping, curl, nmap, netcat).
- Document a firewall policy and justify each rule based on security requirements.

3. Background Knowledge

Review the following concepts before you begin. You will need this understanding to complete the tasks.

3.1 Protocols

A protocol is a set of rules that governs how data is transmitted across a network. The three protocols you will work with most in this lab are:

Protocol	Description	Common Use Cases
TCP	Connection-oriented; guarantees delivery via a three-way handshake.	HTTP/HTTPS (web), SSH (remote access), FTP (file transfer), SMTP (email).
UDP	Connectionless; faster but no delivery guarantee.	DNS (name resolution), DHCP (IP assignment), streaming media, VoIP.
ICMP	Used for diagnostics and error reporting; not a transport protocol.	ping (echo request/reply), traceroute, destination unreachable messages.

3.2 Ports

A port number identifies a specific service on a host. Think of the IP address as a street address and the port as the apartment number.

Range	Category	Examples
0 – 1023	Well-Known Ports	22 (SSH), 53 (DNS), 80 (HTTP), 443 (HTTPS)

1024 – 49151	Registered Ports	3306 (MySQL), 5432 (PostgreSQL), 8080 (alt HTTP)
49152 – 65535	Dynamic / Ephemeral	Assigned temporarily by the OS for client-side connections.

3.3 Firewall Policy

A firewall policy is a high-level document that defines what traffic should be allowed and what should be blocked. A good policy follows the principle of least privilege: deny everything by default and only permit traffic that is explicitly needed.

Every firewall rule typically specifies:

- Direction: Inbound (incoming) or Outbound (outgoing).
- Protocol: TCP, UDP, or ICMP.
- Port(s): The destination port number or range.
- Source / Destination: IP address or subnet.
- Action: ALLOW or DENY.

3.4 UFW vs. iptables

UFW (Uncomplicated Firewall) is a user-friendly front-end for iptables. It simplifies common tasks but hides some of the underlying power. iptables gives you full control over the Linux netfilter framework. In this lab you will use both so you can see how they relate to each other.

4. Prerequisites

- A computer with at least 4 GB of RAM and 20 GB of free disk space.
- A hypervisor installed. VirtualBox (free, recommended) or VMware Workstation Player — you may use any hypervisor you are comfortable with.
- An Ubuntu Server 22.04 LTS ISO (download from ubuntu.com).
- Basic familiarity with the Linux command line (cd, ls, nano/vim, sudo).

5. Project Tasks

Part A — Set Up Your Virtual Machine

1. Create a new VM in your hypervisor with the following minimum specs: 1 CPU, 2 GB RAM, 20 GB disk, bridged or NAT network adapter.
2. Install Ubuntu Server 22.04 LTS. During installation, enable the OpenSSH server when prompted.
3. After installation, log in and run the following commands to update the system:

```
sudo apt update && sudo apt upgrade -y
```

4. Install the tools you will need for testing:

```
sudo apt install -y nmap netcat-openbsd curl net-tools
```

5. Verify your network is working:

```
ip addr show          # Note your VM's IP address
ping -c 3 8.8.8.8     # Test internet connectivity
ping -c 3 google.com  # Test DNS resolution
```

Deliverable A: Take a screenshot showing your VM's IP address and successful ping results.

```
mohanad@firewall-lap:~$ ip addr show && ping -c 3 google.com
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:8d:17:06 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c298d1706
    inet 192.168.142.130/24 metric 100 brd 192.168.142.255 scope global dynamic ens33
       valid_lft 1057sec preferred_lft 1057sec
    inet6 fe80::20c:29ff:fe8d:1706/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
PING google.com (142.250.178.14) 56(84) bytes of data.
64 bytes from ncmrsb-am-in-f14.1e100.net (142.250.178.14): icmp_seq=1 ttl=128 time=77.2 ms
64 bytes from ncmrsb-am-in-f14.1e100.net (142.250.178.14): icmp_seq=2 ttl=128 time=77.0 ms
64 bytes from ncmrsb-am-in-f14.1e100.net (142.250.178.14): icmp_seq=3 ttl=128 time=79.3 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 76.990/77.828/79.326/1.061 ms
mohanad@firewall-lap:~$
```

Part B — Design Your Firewall Policy

Before touching any firewall commands, write a policy document. Use the table below as a template. Your policy must include at least six rules that cover the following scenarios:

- Allow SSH access (port 22) from your host machine only.
- Allow HTTP (port 80) and HTTPS (port 443) inbound for a web server.
- Allow DNS queries (port 53, UDP) outbound.
- Block all other inbound traffic (default deny).
- Allow established and related connections (stateful filtering).
- One custom rule of your choice (e.g., allow MySQL from a specific subnet, block ICMP, rate-limit SSH).

Policy Template:

#	Direction	Protocol	Port(s)	Source / Dest	Action	Justification
1	Inbound	TCP	22	192.168.1.0/24	ALLOW	SSH admin access
2	Inbound	TCP	80, 443	Any	ALLOW	Web server traffic
3	Outbound	UDP	53	Any	ALLOW	Allow outbound DNS queries for domain name resolution.
4	Inbound	Any	Any	Any	DENY	Default deny policy to block all unauthorized inbound traffic.
5	Inbound	TCP/UDP	Any	Any	ALLOW	Allow established and related connections (stateful filtering).

6	Inbound	TCP	3306	192.168.142.0/24	ALLOW	Custom rule to allow MySQL database access from a specific subnet.

Deliverable B: A completed policy table with at least six rules and a justification for each.

Part C — Implement with UFW (Beginner-Friendly)

UFW is the easiest way to get started. Implement your policy using the commands below as a guide.

1. Reset UFW to a clean state and set default policies:

```
sudo ufw reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

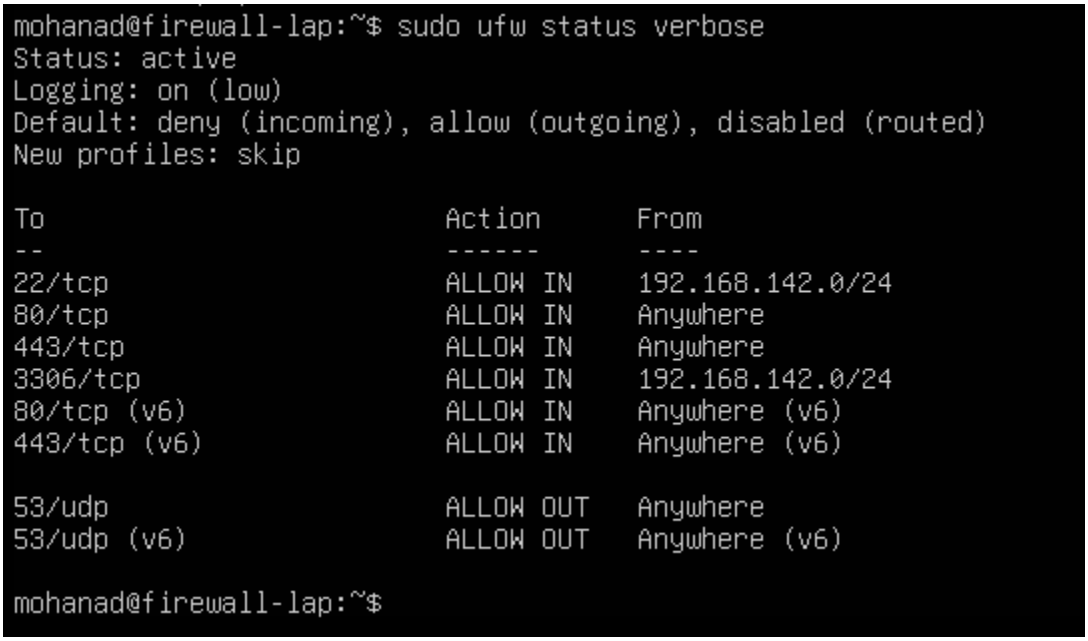
2. Add your rules one at a time. Examples:

```
# Allow SSH from your local network only
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp

# Allow HTTP and HTTPS from anywhere
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Allow outbound DNS
sudo ufw allow out 53/udp
```

3. Enable the firewall and verify:

```
sudo ufw enable
sudo ufw status verbose
```

Deliverable C: A screenshot of the output of “ufw status verbose” showing all your rules active.



Part D — Implement with iptables (Advanced)

Now replicate the same policy using raw iptables commands. This gives you direct visibility into how the Linux kernel filters packets.

1. Disable UFW so it does not conflict:

```
sudo ufw disable
```

2. Flush all existing rules and set default chain policies:

```
sudo iptables -F
sudo iptables -X
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
```

```
sudo iptables -P OUTPUT ACCEPT
```

3. Allow loopback traffic (essential for local services):

```
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT
```

4. Allow established and related connections (stateful rule):

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

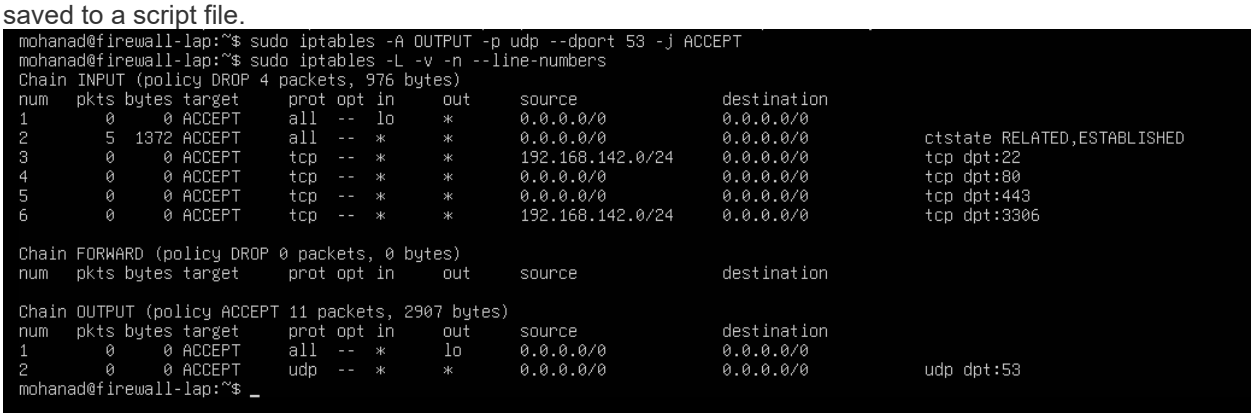
5. Add your specific rules. Examples:

```
# Allow SSH from local network
sudo iptables -A INPUT -p tcp -s 192.168.1.0/24 --dport 22 -j ACCEPT

# Allow HTTP and HTTPS
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Allow outbound DNS
sudo iptables -A OUTPUT -p udp --dport 53 -j ACCEPT
```

6. View your complete ruleset:

```
sudo iptables -L -v -n --line-numbers
```

Deliverable D: A screenshot of the full iptables listing with line numbers, plus your iptables commands saved to a script file.



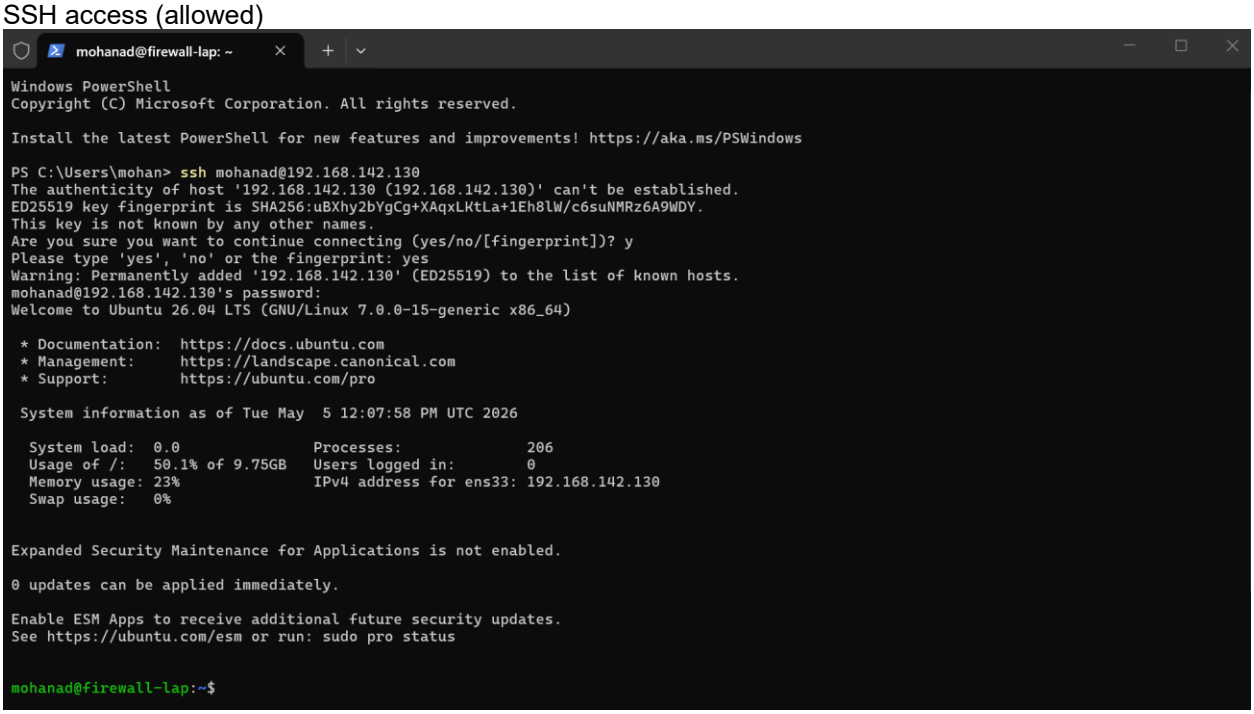
Part E — Test Your Firewall

Testing is critical. For each rule, verify that allowed traffic passes and blocked traffic is dropped.

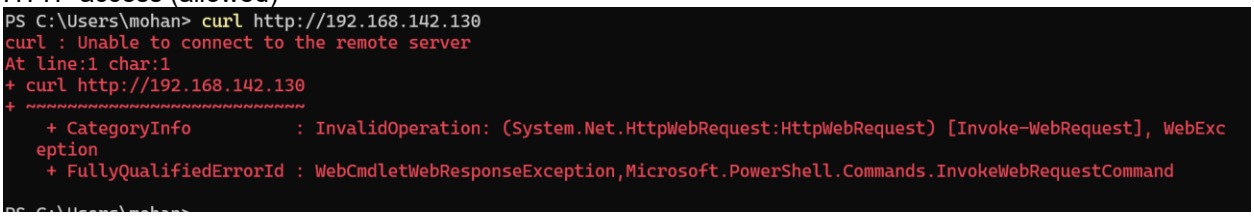
What to Test	Command (from host or second VM)	Expected Result
SSH access (allowed)	`ssh user@<VM-IP>`	Connection succeeds
HTTP access (allowed)	`curl http://<VM-IP>`	Response received (or connection refused if no web server)
Blocked port (e.g., 3306)	`nmap -p 3306 <VM-IP>`	Port shown as filtered/closed
ICMP ping (depends on your policy)	`ping <VM-IP>`	Replies if allowed; timeout if blocked
Full port scan	`nmap -sS <VM-IP>`	Only allowed ports appear as open

Deliverable E: A testing log that shows the command, its output, and whether the result matches your policy (pass/fail) for each rule.

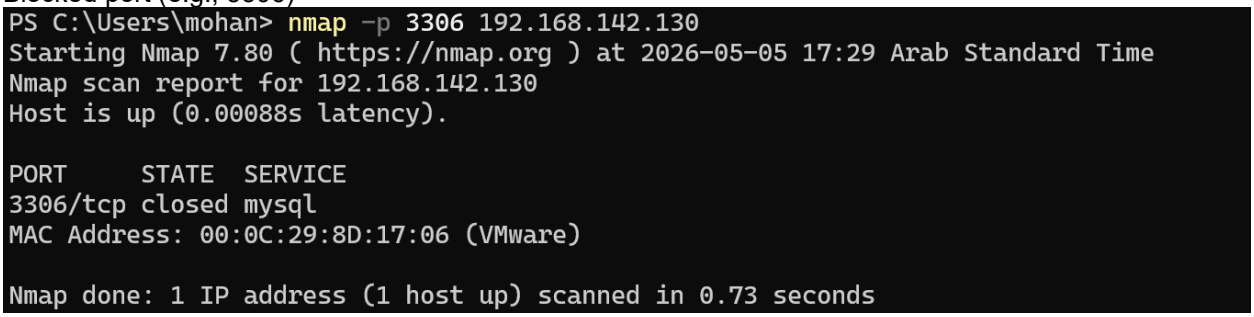
SSH access (allowed)



HTTP access (allowed)



Blocked port (e.g., 3306)



ICMP ping (depends on your policy)

```
PS C:\Users\mohan> ping 192.168.142.130

Pinging 192.168.142.130 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.142.130:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Full port scan

```
PS C:\Users\mohan> nmap -sS 192.168.142.130
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-05 17:31 Arab Standard Time
Nmap scan report for 192.168.142.130
Host is up (0.00s latency).
Not shown: 996 filtered ports
PORT     STATE  SERVICE
22/tcp   open   ssh
80/tcp   closed http
443/tcp  closed https
3306/tcp closed mysql
MAC Address: 00:0C:29:8D:17:06 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 5.93 seconds
```

Part F — Reflection & Documentation

Write a short report (1–2 pages) that answers the following questions:

- What is the difference between a default-deny and a default-allow firewall policy? Which is more secure and why?
- Compare UFW and iptables: which did you find easier to use? In what scenario would iptables be the better choice?
- Pick one of your rules and explain how it would protect against a specific attack scenario.
- What would you add to your firewall policy if this were a production server?

Deliverable F: Your written reflection as a PDF or Word document or Power Point.

1. Default-Deny vs. Default-Allow Policies
Firewall policies are generally categorized into two fundamental approaches: **Default-Deny** and **Default-Allow**.

- **Default-Deny (Whitelisting):** In this configuration, all incoming and outgoing traffic is blocked by default. Access is only granted to specific ports, protocols, or IP addresses that are explicitly defined in the rule set.
- **Default-Allow (Blacklisting):** This policy permits all traffic to pass through the firewall unless a specific rule exists to block it.

The Superior Security Model: The **Default-Deny** policy is significantly more secure. It follows the **Principle of Least Privilege**, ensuring that only the minimum necessary communication channels are open. By dropping all unauthorized packets, it drastically reduces the attack surface and prevents "shadow" services or unknown vulnerabilities from being exploited. A Default-Allow policy is inherently reactive and dangerous, as it assumes all traffic is safe until proven otherwise.

2. Comparison: UFW vs. iptables

Throughout the implementation of this project, both **UFW (Uncomplicated Firewall)** and **iptables** were utilized to secure the Ubuntu Server.

- **Ease of Use: UFW** is considerably easier to use due to its simplified, human-readable command syntax (e.g., ufw allow 22). It acts as a "wrapper" that manages the underlying complex netfilter rules, making it ideal for quick setups and basic server protection.
- **When iptables is the Better Choice: iptables** is the superior choice for high-complexity environments or production-grade infrastructure. It allows for **granular control** over the Linux kernel's packet filtering hooks. Iptables is required when:
    - Implementing complex **Stateful Inspection** rules (e.g., managing RELATED,ESTABLISHED states).
    - Configuring specific **Network Address Translation (NAT)** or port forwarding.
    - Defining rules based on specific network interfaces or advanced packet matching criteria that UFW may not expose.

3. Specific Rule Analysis: SSH Protection

Consider the following rule implemented in the **iptables** configuration: sudo iptables -A INPUT -p tcp -s 192.168.142.0/24 --dport 22 -j ACCEPT

- **Attack Scenario: External Brute Force / Unauthorized Remote Access.**
- **How it Protects:** This rule restricts SSH access specifically to the local subnet (192.168.142.0/24). If an attacker attempts to connect to the server from an external IP address (outside the authorized range), the firewall will automatically drop the packets because they do not match the source IP criteria. Even if an attacker possesses the correct credentials, they cannot reach the login prompt unless they are physically or logically within the trusted network segment.

4. Production Server Enhancements

If this configuration were deployed on a live production server, the following enhancements would be necessary to ensure robust security:

- **Intrusion Prevention Systems (IPS):** Integration with tools like **Fail2Ban** to automatically update firewall rules and ban IP addresses that exhibit malicious behavior, such as repeated failed login attempts.
- **Logging and Auditing:** Adding a LOG target to the iptables script to record dropped packets. This provides visibility into ongoing attack patterns and helps in forensic investigations.
- **Rate Limiting:** Implementing the limit module in iptables to restrict the frequency of incoming connections. This is a critical defense against **Denial of Service (DoS)** attacks and automated scanning tools.
- **Port Knocking:** A stealth mechanism where ports remain closed until a specific sequence of "knocks" (connection attempts to closed ports) is received, further hiding administrative services from public view.

6. Helpful Resources

- Ubuntu UFW documentation: help.ubuntu.com/community/UFW
- iptables tutorial: netfilter.org/documentation
- Common ports reference: iana.org/assignments/service-names-port-numbers
- nmap basics: nmap.org/book/man.html
